Implement **topAbsDiff(nums, S)** that takes a positive integer array **nums[]** and an integer **S**. From the array, pair all even numbers with all odd numbers **in ascending order** (i.e smallest even with smallest odd, second-smallest even with second-smallest odd, and so on), compute the absolute difference for each pair, and **return the top S absolute differences in descending order** in an array

Constraints:

- The array is guaranteed to have an equal number of even and odd values
- No ties among absolute differences
- S ≤ number of **odd-even** pairs
- **No sorting allowed at any stage**
- Only **len() / .length** and **abs() / Math.abs()** inbuilt functions allowed
- You can create an extra array **only to return the desired outputs**

You are provided a **MinHeap** and **MaxHeap** class, each with **insert(val), extractMin() / extractMax()** methods. Heap entries are single integer values. **You can make as many instances of whichever heap you feel necessary.**

Sample parameter	Output	Explanation
S = 2 **nums** = [7, 2, 5, 12, 3, 8]	[5, 3]	Evens ascending: 2, 8, 12 Odds ascending: 3, 5, 7 Absolute diff: |2-3|=1 |8-5|=3 |12-7|=5 Top 2: **5, 3**
S = 3 **nums** = [22, 5, 2, 49, 36, 1, 10, 15]	[13, 7, 5]	Evens ascending: 2, 10, 22, 36 Odds ascending: 1, 5, 15, 49 |2-1|=1 |10-5|=5 |22-15|=7 |36-49|=13 Top 3: **13, 7, 5**

Do not write code for the existing classes or methods; use heaps to implement:

Java	Python

```java
public static int[] topAbsDiff(
            int[] nums, int S) {
    //ToDo
```

```python
def topAbsDiff(nums, S):

    #ToDo
```

Short Questions and MCQ: **[6 * 1 = 6]**

1. A BST is made by consecutively inserting 15, 25, 8, 5, 30, 12, 20, 35, 4, 2, 7, 28. Answer I and II based on this question:

   I. After removing node 8 using inorder predecessor, what is the new left child of 15?
   A. 7 **B. 8** **C. 5** **D. 4**

   II. After removing node 25 using inorder successor, which node moves up to replace 25?
   A. 20 **B. 30** **C. 28** **D. 35**

2. A BST has 167 nodes and a height of 10. What is the minimum number of comparisons needed to confirm a value is absent?

A. 8 **B. 10** **C. 9** **D. 11**

3. Which traversal of a BST outputs node values in descending order?

A. Inorder **B. Postorder** **C. Level-order** **D. None**

4. In a BST, where is the inorder predecessor of a node always found?

A. Rightmost node of left subtree **B. Leftmost node of right subtree**
C. The node's parent **D. Leftmost node of left subtree**

5. A BST can have a root with no left subtree. **True / False**